

QUARTERLY AWARDS • INNOVATION

From undocumented Slate to React, with AI

How we rebuilt the ERM Risk & Opportunity Dashboard on a tight timeline we set ourselves, and added new features on the way

[Team name] • [Presenter names]

A large app that nobody had documented

Step 2: AI extracts every feature

Each module mined for widgets, events, queries and actions

105

features found

Shell app

Print preview

Action tracker

Full search

Heatmap

One-pager

Report table

Waterfall



8

Slate exports: 1 shell app and 7 modules

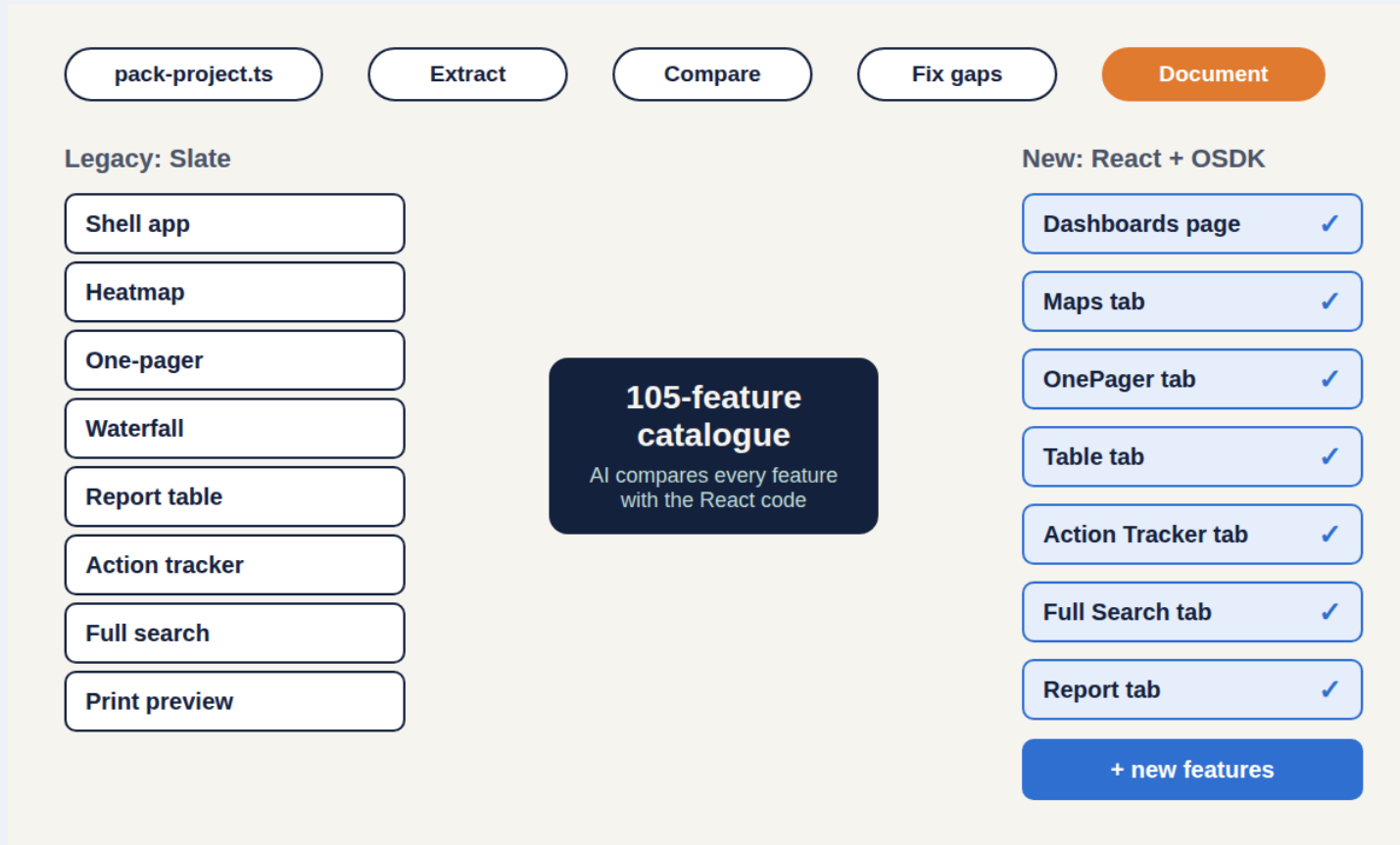
0

pages of documentation, about 25 MB of configuration

105

features recovered by AI and confirmed by users

Our method: AI drafts, people check



pack-project.ts

Our script packs the whole React app, 75 files, into one dated file the AI reads in one go

1. Pack the context
2. Extract legacy features
3. Compare with the React code
4. Fix gaps, re-pack, re-check
5. Document the new app

The process in numbers

1 PI

Whole migration done in
one SAFe Program
Increment

105

legacy features recovered,
with no docs

8

Slate exports analysed,
about 25 MB

75→1

React files packed into one
AI context

31

gaps found and prioritised
by AI review

7

security and data issues
caught in review

5

enhancements beyond the
old app

0→10

documents: 8 diagrams
and 2 guides

Effort saved: about 76% versus manual

~17

Manual estimate

sprints, $\approx$ 340 person-days

4

Actual with AI

sprints, in one PI

~76%

Effort saved

$(17 - 4) \div 17$ sprints

Where the AI saved the most time

Discovery

105 features catalogued from the exports, with no docs to read

Gap analysis

Every feature checked against the packed React code

Documentation

8 architecture and lifecycle diagrams produced alongside the work

Generic estimate, not a measured baseline: 105 features $\times$ (0.5 day discovery + 2 days build + 0.5 day parity testing) + 25 days documentation $\approx$ 340 person-days. Team of 2 developers, 2-week sprints = 20 person-days per sprint; 1 PI = 4 development sprints.

What we delivered in one SAFe PI

105 features

Catalogued and compared one by one, so nothing was lost by accident

Gaps fixed

Including dashboard sharing access in the backend workflow, now deployed

New features

[List the new React features you will demo]

Documented

Architecture and lifecycle diagrams for both apps, where none existed

Our contribution to the 2026 objectives

OUR PROJECT

AI-assisted migration of ERM

Slate to React + OSDK
2 developers · 1 PI
pack-project.ts

EVIDENCE

AI read 8 Slate exports and the packed React code:
105 features recovered, 31 gaps found

Delivered in 1 PI by 2 developers, about 76% less
effort than a manual estimate

All 105 features tracked for parity; ERM moved off
legacy Slate onto a standard, maintainable stack

pack-project.ts and the playbook let any team
repeat the method

2026 OBJECTIVE

Leverage AI to solve business problems

India TFO · Digital: democratise AI

Deliver on time, on cost, on quality

Digital: optimise our digital resources

Ensure our operational performance

Digital: availability of systems for operations

Grow digital acumen and competencies

Digital: develop our people · India TFO

A playbook any team can reuse

Built together

- Business users confirmed the feature catalogue
- Data and security owners kept restricted views intact
- Backend and frontend fixes shipped together

Ready to transfer

- Other Slate apps in [business areas]
- Any low-code tool with little documentation
- Reusable as is: pack-project.ts, comparison template, diagrams

Pack. Extract. Compare. Fix. Document. Repeat for the next app.